Week 19 (Days 127-133)

Status: Days 127-133 (Month 5, policy-gradient RL, exploration, reward shaping and the RL-for-LLMs link-back) are written in full. Every number comes from `code/week-19/day127_*.py` to `day133_*.py`, each run once on this box (CPU only, `torch.set_num_threads(1)`, torch 2.15 CPU, gymnasium 1.4.0), with logs in `code/week-19/out/`. Shared helpers are in `code/week-19/_pg.py`.

Compute note. Everything here uses 4-64-64-2 MLPs on CartPole-v1, tabular grids, or bandits with 5-20 arms. A GPU would not speed these up: the cost is the Python loop that steps the environment one transition at a time, not matrix maths. What a GPU (or several) changes is the Day 132 setting: PPO/GRPO on a real language model, where every "action" is a token from a 50k-way softmax, a rollout is autoregressive generation, and you hold a policy, a reference model and (for PPO) a value model in memory at once. None of that is run here; Day 132 uses a 5-answer toy so that the exact optimum is known.

Sources used for Day 132 (all opened and checked): PPO, https://arxiv.org/abs/1707.06347; InstructGPT, https://arxiv.org/abs/2203.02155; DPO, https://arxiv.org/abs/2305.18290; DeepSeekMath (introduces GRPO), https://arxiv.org/abs/2402.03300.

Day 127: Policy gradients: REINFORCE and baselines

Review today: Day 126 (+1), Day 124 (+3), Day 120 (+7), Day 106 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Why learn the policy directly? Week 18 learned values (V, Q) and acted greedily on them. That works when actions are few and discrete and a max over them is cheap. Policy-gradient methods instead parameterise a stochastic policy pi_?(a|s), usually a softmax over network outputs, and climb the expected return J(?) = E_?~pi_?[G(?)] directly. Advantages: they handle continuous or huge action spaces (a language model's vocabulary is the action space on Day 132), they can represent stochastic optimal policies, and small parameter changes give small behaviour changes instead of the sudden argmax flips that destabilised DQN.

The log-derivative trick. We cannot differentiate through the environment, but we can differentiate the probability of the actions we took:

?? *J = E[Sum_t ??* log pi_?(a_t|s_t) . G_t],

where G_t = Sum_k gamma^k r_{t+k+1} is the return from step t onward. The environment dynamics drop out because they do not depend on ?. In code, you build the "surrogate loss" L = -Sum_t log pi_?(a_t|s_t).G_t, with G_t treated as a constant, and call backward. Only rewards *after* step t multiply step t's log-probability ("reward to go"), because an action cannot affect the past; using the full-episode return is still unbiased but noisier.

Variance is the enemy. The estimator is unbiased but extremely noisy: one episode on CartPole can last 9 steps or 500, and every log-probability in it is pushed by that return. On CartPole all rewards are +1, so every G_t is positive, and REINFORCE pushes up *every* action taken; it only learns because long episodes push harder.

Baselines. For any function b(s) that does not depend on the action, E[? log pi(a|s).b(s)] = b(s).? Sum_a pi(a|s) = b(s).?1 = 0. So we can subtract a baseline without adding bias: use (G_t - b(s_t)). A good b makes the weight positive for better-than-usual actions and negative for worse-than-usual ones, which cuts variance. Two common choices: a constant (the mean return), or a learned value network V_w(s) fitted by regression to G_t. The quantity G_t - V(s_t) is an estimate of the **advantage** A(s,a) = Q(s,a) - V(s), the central object of the rest of the week.

Practical details. Normalising returns per batch is common but changes the effective step size; we keep raw returns and use Adam. Discount gamma 0.99 makes the horizon about 100 steps. Evaluation uses the greedy policy (argmax) on 20 held-out seeds, because the stochastic training policy understates what the network knows.

Block 2: Worked example (45 min)

Run `python code/week-19/day127_reinforce.py` (a few seconds per seed). It first measures the spread of the gradient estimate at initialisation over 20 single-episode estimates, then trains 300 episodes per seed (Adam 1e-3, gamma 0.99) with and without a

learned value baseline.

Verified (out/day127_reinforce.log): - Gradient standard deviation at init: 0.436 with no baseline, 0.324 with a constant mean-return baseline. - No baseline: mean episode length in episodes 1-50 / 251-300 was 18.2 / 159.9 (seed 0), 25.6 / 89.8 (seed 1), 27.4 / 29.1 (seed 2); greedy eval 125.3, 176.6, 29.2. - Value baseline: 19.4 / 218.6, 25.0 / 411.2, 30.9 / 368.2; greedy eval 127.0, 478.9, 308.8. - Environment steps used: 17,397 / 19,571 / 8,383 without a baseline, 45,666-48,298 with one. Each seed ran in 1-4 s.

Reading it. The baseline cut gradient spread by about a quarter, and every seed with a value baseline ended better than its no-baseline twin. But note the confound: a fixed *episode* budget means better policies consume more *steps* (longer episodes), so the baseline runs also saw about 2-5 times more data. Day 133 fixes this by giving every method the same step budget. Seed 2 without a baseline never took off (29.1 at the end): a reminder that three seeds are a minimum, not a proof.

Block 3: Lab (2h)

1. Read `_pg.py`: `mlp`, `discounted`, `evaluate`. Check `discounted([1,1,1], 0.5)` by hand (1.75, 1.5, 1).
2. In `day127_reinforce.py`, replace reward-to-go with the full-episode return for every step. Predict the effect on gradient spread, then measure it with the same 20-episode probe.
3. Add return normalisation (subtract mean, divide by std within an episode). Compare final greedy eval across the same three seeds and record it in your notes.
4. Plot episode length against episode index for both variants (matplotlib, saved to `out/`).
5. Write three sentences on why seed-to-seed spread is so large here.

Block 4: Practice (1h)

1. Prove that a state-dependent baseline adds no bias.
2. Why does every CartPole action get pushed up when there is no baseline?
3. Why did the baseline runs use more environment steps?
4. Give one reason to prefer reward-to-go over the full return.
5. What does a negative (G_t - V(s_t)) do to the probability of a_t?

Answers. 1) E_a[?log pi.b] = b.Sum_a ?pi = b.?1 = 0. 2) Every return is positive, so every log-probability gets a positive weight. 3) They learned longer episodes, and the budget was counted in episodes. 4) It drops terms that are pure noise (rewards before t), lowering variance. 5) It decreases it.

Block 5: Review (1h)

Do your review days (listed above). Then write, from memory, the REINFORCE gradient, the baseline argument and the advantage definition. Make five flashcards: log-derivative trick; reward to go; baseline unbiasedness; advantage; why greedy evaluation.

Extra notes

- REINFORCE is on-policy: each episode is used once and thrown away. This is why it is sample-hungry, and why PPO (Day 129) re-uses each batch several times under a safeguard.
- The value network is trained with plain MSE on G_t. It does not need to be accurate to help; it only needs to be correlated with the return.
- Entropy bonuses (Day 128) are another way to stop the softmax collapsing too early.

Going deeper

Read the policy-gradient chapter of Sutton & Barto's *Reinforcement Learning: An Introduction* (2nd ed.) (the authors host a free PDF; find it from their site), focusing on the REINFORCE-with-baseline box. Compare its update with the code line by line.

Interview-style question (15 min)

"Your REINFORCE agent learns on 2 of 3 seeds. How do you decide whether the fix is a baseline, a smaller learning rate or more seeds?" Strong answer: measure gradient variance at a fixed policy (as in Block 2), run at least 5 seeds per setting under a fixed step budget, look at learning curves rather than final numbers, and change one thing at a time.

Common mistakes

- Letting gradients flow into G_t or into the baseline inside the policy loss (detach them).
- Comparing methods under an episode budget, which favours whichever learns first.
- Reporting the stochastic training return as "performance" when the greedy policy is better (or worse).
- Drawing conclusions from one seed.

What to do next

Tomorrow replaces the Monte Carlo return with a bootstrapped one: the critic stops being just a baseline and becomes part of the target. Skim your Day 123 notes on TD targets first.

Day 128: Actor-critic (A2C)

Review today: Day 127 (+1), Day 125 (+3), Day 121 (+7), Day 107 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

From baseline to critic. In REINFORCE with a baseline, V(s) only reduces variance; the target is still the full Monte Carlo return, so the agent must wait until the episode ends. Actor-critic methods use the value function *inside the target*. The one-step advantage estimate is the TD error

?*t* = *r*{t+1} + gamma V(s_{t+1}) - V(s_t),

and an n-step version sums n real rewards before bootstrapping: A_t ~= r_{t+1} + gamma r_{t+2} + ... + gamma^{n-1} r_{t+n} + gamma^n V(s_{t+n}) - V(s_t). Small n means low variance but biased by the critic's errors; large n moves back towards Monte Carlo. This is the same bias-variance dial you met with TD(0) vs MC on Day 123, now applied to the policy gradient.

The two losses. The actor minimises -log pi(a_t|s_t).A_t (A detached). The critic minimises (target - V(s_t))^2. Most implementations add an **entropy bonus** -c.H(pi(.|s)) to the actor loss to keep exploring; we use c = 0.01. A2C ("advantage actor-critic") is the synchronous form: collect n steps, compute advantages, take one gradient step, repeat. Because updates happen every n steps instead of every episode, learning starts immediately.

Truncation versus termination. CartPole-v1 stops at 500 steps by *truncation*, not because the pole fell. If you treat that as terminal (V = 0 afterwards), you teach the critic that standing upright at step 500 is worthless. Gymnasium returns `terminated` and `truncated` separately; we bootstrap through truncation and only zero the value on true termination. This is a small line of code with a large effect on stability.

Why A2C is fragile. The actor follows a critic that is itself learning, and the target moves with both. One bad step can push the policy into a region where episodes are short, the critic learns those, and the agent does not recover. You will see exactly that in the log.

Block 2: Worked example (45 min)

Run `python code/week-19/day128_actor_critic.py`: A2C, 50k environment steps, entropy 0.01, Adam 1e-3, comparing n = 1 and n = 32, three seeds each, with greedy evaluation every 10k steps.

Verified (out/day128_actor_critic.log):

n	Seed	Episodes	Mean length, last 20	Greedy eval at 10k / 20k / 30k / 40k / 50k	Time
1	0	289	377.6	245.8 / 500.0 / 225.2 / 210.9 / 443.5	53 s
1	1	235	495.4	500 / 396.6 / 255.4 / 500 / 500	53 s
1	2	2,117	260.1	404.9 / 9.3 / 9.3 / 500 / 500	53 s
32	0	242	440.6	126.5 / 281.7 / 380.2 / 500 / 500	7 s

n	Seed	Episodes	Mean length, last 20	Greedy eval at 10k / 20k / 30k / 40k / 50k	Time
32	1	293	336.1	401.1 / 500 / 143.1 / 188.8 / 500	7 s
32	2	314	264.5	500 / 167.7 / 242.9 / 223.9 / 462.9	7 s

Reading it. All six runs reached 500 at some checkpoint, much faster than REINFORCE in steps. But the curves are not monotone: n = 1 seed 2 collapsed mid-run to 9.3 (the pole falls almost at once), which is why it logged 2,117 episodes, then recovered. n = 32 ran about 7 times faster in wall-clock time because it makes one gradient step per 32 transitions instead of one per transition. Its final numbers are not clearly better or worse at three seeds. The lesson: "solves CartPole" and "is stable" are different claims.

Block 3: Lab (2h)

1. Change truncation handling to "treat as terminal" and rerun n = 32 for one seed. Compare evaluation curves.
2. Set the entropy coefficient to 0 and to 0.05; record the final greedy eval per seed.
3. Add a print of the mean policy entropy every 10k steps. Does the collapse in n = 1 seed 2 coincide with an entropy drop?
4. Try n = 8. Where does it sit in time and stability?

Block 4: Practice (1h)

1. Write the n-step advantage.
2. Which n is closer to Monte Carlo?
3. Why bootstrap on truncation?
4. Why is n = 32 faster in wall-clock time?
5. What does the entropy bonus prevent?

Answers. 1) Sum_{k<n} gamma^k r_{t+k+1} + gamma^n V(s_{t+n}) - V(s_t). 2) Larger n. 3) The state is not terminal; its true value is not 0. 4) Fewer gradient steps per transition. 5) Premature collapse to a near-deterministic policy.

Block 5: Review (1h)

Do your review days. Redraw the actor-critic loop from memory (environment -> buffer of n steps -> advantages -> two losses -> step). Flashcards: TD error as advantage; n-step trade-off; entropy bonus; terminated vs truncated.

Extra notes

- Sharing one torso between actor and critic is common; we use two separate MLPs to keep the losses independent.
- A3C ran many asynchronous workers; A2C's synchronous version is simpler and, on CPUs, usually as good.
- GAE (Day 129) mixes all n-step estimates with a weight lambda instead of picking one n.

Going deeper

Re-derive the one-step actor-critic update in Sutton & Barto (the book cited on Day 127) and compare it to the code's `train` function.

Interview-style question (15 min)

"Your actor-critic reaches 500 and then collapses to 9. What do you check?" Look at policy entropy, critic loss and advantage scale at the collapse; check terminal/truncation handling; lower the learning rate or add gradient clipping; and consider a trust-region-style method (tomorrow).

Common mistakes

- Forgetting to detach the advantage in the actor loss (the actor then trains the critic).
- Bootstrapping across a true termination.
- Reporting only the best checkpoint.

What to do next

Tomorrow: PPO, which keeps actor-critic but limits how far each update can move the policy, and re-uses data for several epochs.

Day 129: PPO from scratch

Review today: Day 128 (+1), Day 126 (+3), Day 122 (+7), Day 108 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

The problem PPO solves. A2C takes one gradient step per batch and throws the batch away. Taking many steps on the same batch would be more sample-efficient, but the policy-gradient estimate is only valid near the policy that collected the data. Push too far and the update is computed from data that no longer describes your policy, which is how runs collapse.

Importance ratio. Let pi_old be the policy that collected the batch and r_t(?) = pi_?(a_t|s_t) / pi_old(a_t|s_t). At ? = ?_old the gradient of E[r_t.A_t] equals the ordinary policy gradient, so r_t.A_t is a surrogate objective we can optimise for several epochs.

Clipping. PPO (Schulman et al. 2017, https://arxiv.org/abs/1707.06347) maximises

L^CLIP = E[min(r_t A_t, clip(r_t, 1-eps, 1+eps) A_t)].

Read the two cases. If A_t > 0, the objective stops increasing once r_t > 1+eps: no incentive to make a good action much more likely than before in one update. If A_t < 0, it stops once r_t < 1-eps. The min makes this a pessimistic bound: clipping can only remove incentive, never add it. It is a cheap stand-in for a trust region (a KL constraint between old and new policy), and you can monitor that KL directly.

GAE. PPO usually computes advantages with generalised advantage estimation: A_t = Sum_k (gammalambda)^k ?_{t+k}, where ? is the TD error from Day 128. lambda = 0 is the one-step TD advantage; lambda = 1 is the Monte Carlo advantage minus V. We use gamma 0.99, lambda 0.95, and the recursion runs backwards over the rollout, resetting at true terminations.

The full loop. Collect T = 1024 steps with pi_old; compute GAE and value targets (A + V); normalise advantages; for 10 epochs, shuffle into minibatches of 64 and step on policy loss + 0.5.value loss - 0.01.entropy, with gradient-norm clipping at 0.5 and Adam 3e-4; repeat until 50k steps. Diagnostics after each update: approximate KL between old and new policy, and the fraction of samples whose ratio left [0.8, 1.2].

Block 2: Worked example (45 min)

Run `python code/week-19/day129_ppo.py` (about 30 s per seed). It trains with eps = 0.2 and then an ablation with no clipping (eps = 10^9: the same 10 epochs of plain importance-weighted policy gradient on each batch).

Verified (out/day129_ppo.log): - **Clip 0.2:** seed 0 had 326 episodes and its greedy eval was 500 from update 30 onward; seed 1, 276 episodes, first 500 at update 16 and 500 from about update 29 onward; seed 2, 317 episodes, 500 from about update 32 onward. Approximate KL per update: max 0.017-0.019, mean 0.006-0.007. About 0.06 of ratios were outside [0.8, 1.2]. 29-30 s per seed. - **No clip:** 619, 456 and 624 episodes. KL max 2.432 / 2.790 / 2.352, mean 0.531 / 0.607 / 0.339; clip fraction 0.53 / 0.55 / 0.47; 26 s per seed. Evaluations kept collapsing: seed 0 fell to 10, 10, 10 at updates 44-46 before ending at 500; seed 1 ended at 121; seed 2 hit 39, 10, 14 at updates 45-47 and ended at 476.

Reading it. With clipping, the per-update KL stayed below 0.02 and all three seeds held 500 for the last ~17-20 updates: that is the stability A2C lacked on Day 128. Without clipping, the KL per update was about 100 times larger and half the samples had moved outside the clip range; the policy repeatedly jumped to near-random behaviour (10 steps is roughly how long a falling pole lasts). The higher episode count for "no clip" is not a sign of learning more: short failed episodes inflate it. Wall-clock time is higher than A2C because of 10 epochs of minibatch updates, but the samples are used 10 times.

Block 3: Lab (2h)

1. Try eps = 0.1 and eps = 0.3 for one seed; log KL and the update at which eval first holds 500.
2. Set epochs to 1 with eps = 0.2. How close is this to A2C with n = 1024?
3. Replace clipping by early stopping: break the epoch loop when approx KL > 0.02. Compare stability.
4. Remove advantage normalisation and see what happens to the clip fraction.

Block 4: Practice (1h)

1. Write L^CLIP and explain both signs of A.
2. What is r_t at the first minibatch of the first epoch?
3. What lambda makes GAE equal to the TD(0) advantage?
4. Why does clipping alone not guarantee a small KL?
5. Why is the "no clip" episode count misleading?

Answers. 1) See Block 1. 2) Exactly 1, since ? = ?_old. 3) lambda = 0. 4) Clipping only removes gradient for samples already outside the range; many epochs and shared parameters can still move other probabilities (the log shows KL, so you can check). 5) Collapsed policies produce many very short episodes.

Block 5: Review (1h)

Do your review days. Write the PPO loop as ten lines of pseudocode from memory. Flashcards: importance ratio; clipped objective; GAE lambda; approx-KL monitor; why multiple epochs need a safeguard.

Extra notes

- "Approximate KL" here is the mean of (log pi_old - log pi_new) over the batch. Day 132 meets a different estimator, used by GRPO, that is always non-negative.
- Value-loss clipping, learning-rate annealing and observation normalisation are common extras; we leave them out to keep the core visible.
- Implementations disagree on many small details, and those details can matter as much as the headline algorithm. Always state your settings.

Going deeper

Read Sections 3 (clipped surrogate) and 5 (algorithm) of the PPO paper (link above) and map each symbol to a line of `day129_ppo.py`.

Interview-style question (15 min)

"Why does PPO clip the ratio instead of the loss or the gradient?" Because the ratio measures how much the policy changed on that sample; clipping it removes the incentive to change further while keeping the ordinary gradient near the old policy. Mention the KL monitor as the honest check that it worked.

Common mistakes

- Recomputing log pi_old after the first epoch (it must be frozen per batch).
- Forgetting the min (clipping alone can increase the objective for negative advantages).
- Resetting GAE at truncation as if it were termination.

What to do next

Tomorrow steps back from deep RL to the simplest exploration problem, the multi-armed bandit, and links it to recommendation.

Day 130: Exploration: bandits, UCB, Thompson sampling and a LinUCB recommendation tie-in

Review today: Day 129 (+1), Day 127 (+3), Day 123 (+7), Day 109 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Bandits are RL with one state. You repeatedly choose one of K arms; arm k pays a reward drawn from an unknown distribution with mean mu_k. There are no states to plan over, so the only difficulty is the **exploration-exploitation** trade-off. Performance is measured by **regret**: Sum_t (mu* - mu_{a_t}), the reward lost compared with always pulling the best arm.

Strategies. - *Greedy:* pull the arm with the highest sample mean. It can lock onto a mediocre arm forever after a lucky start. - *eps-greedy:* with probability eps pull a random arm. Simple, but it keeps exploring at a fixed rate forever, so regret grows linearly. - *Optimistic initial values:* start every estimate at the maximum possible reward (Q0 = 1 for Bernoulli). Each pull disappoints, so the agent tries everything early, then settles. - *UCB1:* pull argmax_k Q_k + c.sqrt(2 ln t / N_k). The bonus shrinks as an arm is tried; this is "optimism in the face of uncertainty" with a guarantee of logarithmic regret. - *Thompson sampling:* keep a Beta(?_k, ?_k) posterior per arm, sample one value from each, pull the arm with the highest sample, update with the observed 0/1 reward. It explores exactly as much as the posterior is uncertain.

Contextual bandits and recommendation. In a recommender, each round you see a context x (user features), choose an item, and observe a click. A contextual bandit learns a model of reward given context and item. **LinUCB** assumes reward ~= ?_k.x for each item k, fits ?_k by ridge regression, and adds an uncertainty bonus ?.sqrt(x^T A_k?^1 x). This is the same optimism idea as UCB1, with the count replaced by a covariance. Without context, a single "best item for everyone" policy cannot exploit the fact that different users like different things.

Why this belongs in an RL week. Every exploration idea in deep RL (entropy bonuses, optimism, posterior sampling, count-based bonuses) has its cleanest form here. And at LLM scale, sampling G answers per prompt and reinforcing the better ones (Day 132) is a bandit-like structure per prompt.

Block 2: Worked example (45 min)

Run `python code/week-19/day130_bandits.py`. Part 1: a 10-armed Bernoulli bandit, 2,000 pulls, 200 independent runs; regret +/- standard error, and the fraction of the last 500 pulls that chose the optimal arm. Part 2: a simulated recommender with 20 items, 5-dimensional user context, 5,000 rounds, 20 runs.

Verified (out/day130_bandits.log):

Policy	Regret (+/- s.e.)	% optimal, last 500
greedy	558.8 +/- 36.0	16%
eps-greedy 0.1	112.7 +/- 2.6	79%
optimistic (Q0 = 1)	38.5 +/- 5.6	77%
UCB1	183.3 +/- 1.9	63%
Thompson	30.5 +/- 1.1	88%
Recommender policy	**CTR**	**Regret**
random	0.382	2832
eps-greedy, no context	0.399	2738
LinUCB (? 1)	0.876	342

Reading it. Thompson sampling had the lowest regret and the highest share of optimal pulls. Greedy was worst by far and has large run-to-run spread (+/- 36.0), the signature of lock-in on a wrong arm. A surprise worth keeping: **UCB1 did worse than eps-greedy at this horizon**. UCB1's bonus is conservative; it keeps sampling arms that are probably bad, and its logarithmic-regret advantage only pays off over longer horizons. Asymptotic guarantees do not rank methods at a finite budget. In the recommender, ignoring context barely beat random (0.399 vs 0.382 CTR), while LinUCB more than doubled CTR: here the context, not the exploration rule, is what matters.

Block 3: Lab (2h)

1. Extend the horizon to 20,000 pulls (fewer runs, e.g. 50, to stay within a few minutes). Does UCB1 overtake eps-greedy?
2. Try UCB1 with a smaller exploration constant; report regret.
3. Make the bandit non-stationary (means drift slowly). Which method suffers most? Add a constant step size to eps-greedy.
4. In the recommender, vary ? in LinUCB (0.1, 1, 3).

Block 4: Practice (1h)

1. Define regret.
2. Why does eps-greedy have linear regret?
3. What replaces the count N_k in LinUCB?
4. Why can optimistic initialisation fail in a non-stationary bandit?
5. Explain the UCB1 result above in one sentence.

Answers. 1) Sum (mu* - mu_{a_t}). 2) It explores at rate eps forever, each time losing on average a fixed amount. 3) The ridge covariance A_k (via x^TA?^1x). 4) Its exploration happens only at the start. 5) Its bonus over-explores at a 2,000-pull horizon, where its log-regret guarantee has not yet paid off.

Block 5: Review (1h)

Do your review days. Flashcards: regret; UCB1 formula; Thompson for Bernoulli; LinUCB bonus; "asymptotic != finite-horizon ranking".

Extra notes

- Off-policy evaluation (estimating a new recommender's CTR from logs of an old one) is the practical bottleneck in real systems; it uses the same importance ratios as PPO.
- Real recommenders face delayed and biased feedback (people only click what they see).

Going deeper

Read the bandit chapter of Sutton & Barto (cited on Day 127) and reproduce its 10-armed testbed figure with `day130_bandits.py`.

Interview-style question (15 min)

"You A/B test a new recommender and it wins slightly. Why might a bandit be better than a fixed 50/50 test, and when would you still use the A/B test?" Bandits shift traffic to the winner and cut regret; a fixed test gives cleaner, unbiased effect estimates and is easier to analyse.

Common mistakes

- Comparing methods on one run (greedy's +/- 36.0 shows why).
- Tuning eps or c on the same runs you report.
- Ignoring context when it is available.

What to do next

Tomorrow: what goes wrong when you change the reward to make learning easier.

Day 131: Reward shaping pitfalls

Review today: Day 130 (+1), Day 128 (+3), Day 124 (+7), Day 110 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Sparse rewards are slow. If the only reward is +1 at the goal, an agent must stumble onto the goal by chance before anything is learned. A natural fix is **reward shaping**: add intermediate rewards that hint at progress. The danger is that the agent optimises the reward you wrote, not the goal you meant.

The classic failure. Reward the agent +0.1 every time it moves closer to the goal, and nothing for moving away. A loop "step closer, step back, step closer..." now earns 0.1 every two steps, forever. With gamma = 0.99 that stream is worth about 0.1/(1-0.99^2) ~= 5, far more than the single +1 at the goal. The optimal policy for the shaped MDP is to never finish. This is **reward hacking** in miniature, and it is the same phenomenon Month 4 met when reward models were exploited by RLHF policies.

Potential-based shaping. Ng, Harada and Russell (1999) showed that if the shaping term has the form F(s, s') = gamma?(s') - ?(s) for any function ? of state, the optimal policy is unchanged. Intuition: along any path the terms telescope, so the total shaping bonus depends only on the start and end states, and loops earn exactly zero (minus discounting). You can encode a heuristic (here ? = -0.1 x Manhattan distance) without creating a loophole. With ?(goal) = 0, the shaped values are V'(s) = V(s) - ?(s).

But "preserves the optimal policy" is not "always faster". Shaping changes the values the learner starts from and the paths it explores. A potential can help on average while still sending some runs down an unlucky road. Always measure.

Block 2: Worked example (45 min)

Run `python code/week-19/day131_reward_shaping.py`. Setup: a deterministic 10x10 grid from (0,0) to (9,9), +1 on reaching the goal, gamma 0.99, a 200-step cap. Two checks: (1) value iteration on each reward, and whether its greedy policy reaches the goal; (2) tabular Q-learning (? 0.5, eps 0.1, Q0 = 0), 10 seeds, recording the first episode at which the greedy policy follows an 18-step shortest path (within 1,000 episodes).

Verified (out/day131_reward_shaping.log): - **Base reward:** the value-iteration policy reaches the goal in 18 steps. Q-learning first-success episodes: 70, 80, 58, 49, 53, 38, 63, 55, 53, 51; 10/10 seeds, median 54. - **Naive shaping (+0.1 per step closer):** the value-iteration policy does **not** reach the goal (it loops). Q-learning: 0/10 seeds ever found the shortest path. - **Potential-based (? = -0.1.distance):** the value-iteration policy reaches the goal in 18 steps. Q-learning: 2, 95, 4, 38, 2, 70, 3, none, 94, 73; 9/10 seeds, median 38.

Reading it. The naive reward did not just slow learning; it changed the optimal behaviour, so even exact dynamic programming learned to loop. Potential-based shaping kept the optimum, and four seeds solved it within 4 episodes, but the spread is huge (2 to 95) and one seed never got there in 1,000 episodes. The median improved (38 vs 54), yet a safe claim from 10 seeds is "faster on most runs, not reliably faster". The base reward was the most consistent.

Block 3: Lab (2h)

1. Add a penalty of -0.1 for moving away (so loops net zero, undiscounted). Does VI now reach the goal? Why is this close to, but not exactly, potential-based?
2. Try ? = -0.5.distance and ? = -0.01.distance; record median and failures.
3. Investigate the failed potential-based seed: print its greedy path at episode 1,000.
4. Write a short note: three real-world reward specifications that could be hacked the same way.

Block 4: Practice (1h)

1. Write the potential-based shaping term.
2. Why does it not change the optimal policy?
3. Estimate the discounted value of the naive loop.
4. Why did VI fail under naive shaping, not just Q-learning?
5. What is the honest summary of the potential-based result?

Answers. 1) F = gamma?(s') - ?(s). 2) Sums telescope; every policy's return shifts by the same -?(s0). 3) About 0.1/(1-gamma^2) ~= 5 (versus 1 for finishing). 4) The optimum of the shaped MDP itself is to loop. 5) Median faster (38 vs 54), high spread, one seed in ten failed.

Block 5: Review (1h)

Do your review days. Flashcards: reward hacking; potential-based shaping; telescoping; "policy-preserving != faster".

Extra notes

- Reward models in RLHF are learned shaping functions with no such guarantee, which is one reason RLHF keeps a KL penalty to a reference model (Day 132).
- Termination rewards and penalties are also shaping: a per-step -1 makes the agent want to end episodes, which can mean failing fast.

Going deeper

Search for Ng, Harada and Russell, "Policy invariance under reward transformations" (ICML 1999), and read the main theorem and its proof sketch.

Interview-style question (15 min)

"Your agent maximises a shaped reward but ignores the real goal. How do you diagnose and fix it?" Evaluate on the true objective separately; look at trajectories, not just returns; replace ad-hoc bonuses with potential-based ones or remove them; and treat any learned reward as exploitable.

Common mistakes

- Evaluating with the shaped reward instead of the true one.
- Rewarding progress without penalising regress.
- Claiming a speed-up from one or two seeds.

What to do next

Tomorrow links all of this back to Month 4: PPO with a KL penalty (RLHF), GRPO, and DPO, on a toy where the answer is known exactly.

Day 132: RL for LLMs, revisited: PPO with a KL penalty, GRPO and DPO

Review today: Day 131 (+1), Day 129 (+3), Day 125 (+7), Day 111 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

The mapping. For a language model, the state is the prompt plus the tokens so far, an action is the next token, and a full answer is an episode. A reward model (or a checker, e.g. for maths answers) scores the finished answer. Everything below is stated only as far as it can be checked in the cited papers.

RLHF with PPO (InstructGPT, https://arxiv.org/abs/2203.02155). The pipeline is supervised fine-tuning, then a reward model trained on human rankings of several responses per prompt (the paper uses K = 4 to 9 responses), then PPO against that reward. The paper adds a per-token KL penalty from the SFT model to the reward, to stop the policy drifting into text that fools the reward model, and initialises the value function from the reward model. A variant, "PPO-ptx", also mixes in pretraining gradients. The DeepSeekMath paper describes the same set-up as the reward r_t = r_? - ?.log(pi_?/pi_ref) per token, citing Ouyang et al. 2022. This is Day 128-129's PPO with Day 131's lesson built in: the learned reward is exploitable, so the KL term is a leash.

The objective being approximated. Maximising E[r] - ?.KL(pi||pi_ref) has the closed-form solution pi*(y) ? pi_ref(y).exp(r(y)/?). You derived and computed this on Day 106; the DPO paper (https://arxiv.org/abs/2305.18290) uses it as its starting point.

DPO. Rearranging the closed form gives r(y) = ?.log(pi*(y)/pi_ref(y)) + ?.log Z. Substituting into the Bradley-Terry preference model makes Z cancel, so the policy can be fitted directly to preference pairs with a logistic loss on ?.(log-ratio of the winner - log-ratio of the loser). The paper's stated advantage is eliminating the need to sample from the LM during fine-tuning: it is offline, with no reward model and no PPO loop. You implemented it on Day 108.

GRPO (DeepSeekMath, https://arxiv.org/abs/2402.03300). The paper describes GRPO as a variant of PPO that drops the critic and instead estimates the baseline from group scores. For each question, sample a group of G outputs, score them, and give each output the advantage (r_i - mean(r)) / std(r); with outcome rewards that advantage is applied to every token of the output. The clipped ratio objective is kept. Instead of putting KL in the reward, GRPO adds ?.KL to the loss, using the estimator pi_ref/pi - log(pi_ref/pi) - 1, which the paper notes is guaranteed to be positive. Reported settings include policy learning rate 1e-6, KL coefficient 0.04 and 64 samples per question. The paper also frames SFT, rejection-sampling fine-tuning, DPO, PPO and GRPO in one "unified paradigm" that differs in data source and gradient coefficient, and reports that RL improved Maj@K but not Pass@K in its experiments.

Connections to this week. GRPO's group mean is Day 127's constant baseline, estimated per prompt; dropping the critic saves a model the size of the policy, which matters at LLM scale. Its group of samples per prompt is a bandit (Day 130) per prompt. The KL leash is the defence against Day 131's reward hacking.

Block 2: Worked example (45 min)

Run `python code/week-19/day132_rl_for_llms.py` (about 4 s). It is a one-prompt "LLM" with five possible answers, reusing Day 106's numbers: rewards [1.0, 0.8, 0.5, 0.2, 0.0], pi_ref = [0.05, 0.1, 0.2, 0.3, 0.35], ? = 0.2. The policy is a softmax over five logits initialised at log pi_ref, trained with Adam (lr 0.05) for 400 iterations, three seeds. Methods: REINFORCE with KL in the reward (64 samples, batch-mean baseline); PPO-clip with KL in the reward (4 epochs, eps 0.2); GRPO (8 groups of 8, group-normalised advantage, clipped, k3 KL in the loss with coefficient ?); DPO (64 pairs per iteration sampled from pi_ref, labelled by Bradley-Terry with the true rewards).

Verified (out/day132_rl_for_llms.log): - Closed form pi* = [0.45, 0.331, 0.148, 0.049, 0.021]; reward 0.799; KL to ref 1.193 (matches Day 106). pi_ref has reward 0.290 and total-variation (TV) distance 0.631 to pi*. - REINFORCE+KL and PPO-clip+KL, all three seeds: pi = [0.45, 0.331, 0.148, 0.049, 0.021], TV 0.000, reward 0.799, KL 1.193. - GRPO: pi ~= [0.921, 0.013, 0.018, 0.021, 0.027] (seed 0), TV 0.476 / 0.478 / 0.463, reward 0.945 / 0.943 / 0.925, KL 2.490 / 2.492 / 2.363. - DPO: TV 0.060 / 0.080 / 0.033, reward 0.796 / 0.794 / 0.815, KL 1.185 / 1.179 / 1.282 (seed 1: pi = [0.391, 0.388, 0.172, 0.034, 0.016]).

Reading it. Both on-policy methods with KL in the reward land exactly on the closed-form optimum, which confirms they optimise E[r] - ?.KL. DPO, trained only on offline pairs from pi_ref, gets close (TV 0.03-0.08); the remaining error is noise from a fixed learning rate on sampled pairs, visible in seed 1 swapping the top two answers' masses. GRPO does **not** converge to the same pi*: it reached higher reward and roughly double the KL. Two reasons follow from its definition, not from any claim in the paper: dividing by the group std rescales the reward, so the effective trade-off against the KL term is not the same ?; and once most groups contain the same answer, their std is near zero and their advantage is zero, so only the KL term acts on them. The takeaway is not "GRPO is wrong" but that **? is not portable between methods**; the same number means a different leash.

Block 3: Lab (2h)

1. Remove the std division in GRPO (use r - mean only). Does it move towards pi*? Report TV.
2. Change the group size to 4 and 32.
3. Put GRPO's KL into the reward instead of the loss. Compare.
4. Halve DPO's learning rate (or decay it) and check whether TV falls.
5. Write a paragraph: which of these four would you choose at LLM scale with one GPU, and why (memory for policy, reference, value and reward models; online sampling cost).

Block 4: Practice (1h)

1. Write the closed-form KL-regularised optimum.
2. In InstructGPT-style PPO, where does the KL go?
3. In GRPO, where does it go and which estimator is used?
4. What does GRPO use instead of a critic?
5. Why does DPO not need sampling during training?

Answers. 1) pi* ? pi_ref.exp(r/?). 2) Into the per-token reward, as -?.log(pi/pi_ref). 3) Into the loss, with pi_ref/pi - log(pi_ref/pi) - 1. 4) The mean (and std) of rewards in a group of samples for the same prompt. 5) It fits the policy directly to fixed preference pairs through the closed-form relation between reward and policy.

Block 5: Review (1h)

Do your review days, especially Day 111's self-check. Flashcards: closed-form optimum; KL in reward vs in loss; group-relative advantage; k3 estimator; DPO's implicit reward; "? is not portable".

Extra notes

- The toy has one prompt and five answers, so there is no credit assignment over tokens, no value model and no reward model error. Those are exactly the parts that make real RLHF hard and expensive.
- On real models these methods need GPUs: the policy, a frozen reference and (for PPO) a value model, plus generation at every step. On this CPU, Month 4's SmolLM2-135M DPO was feasible; online PPO or GRPO on it would be very slow and was not attempted.

Going deeper

Read the GRPO section and the "unified paradigm" section of the DeepSeekMath paper, then Section 4 of the DPO paper. For each, write which line of `day132_rl_for_llms.py` corresponds to which equation.

Interview-style question (15 min)

"Why might a team choose GRPO over PPO for a maths reasoning model?" From the paper: no critic to train or store, and the baseline comes from multiple samples of the same question, which suits verifiable outcome rewards. Mention the costs: G samples per prompt, and a reward-scale change through std normalisation.

Common mistakes

- Claiming two methods "optimise the same objective" because both have a ?.
- Forgetting that DPO's data distribution (here pi_ref) limits what it can learn.
- Quoting hyper-parameters from a paper as if they transfer to your setting.

What to do next

Tomorrow's project compares REINFORCE, A2C and PPO under one honest step budget.

Day 133: PROJECT W19: REINFORCE vs A2C vs PPO on CartPole at an equal step budget; Quiz 19

Review today: Day 132 (+1), Day 130 (+3), Day 126 (+7), Day 112 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

The question. Days 127-129 each used a different budget and evaluation schedule, so their numbers cannot be compared. The project fixes the protocol: every method gets about 50,000 environment steps; five fresh seeds (10-14) that were not used during development, to avoid tuning on the test seeds; and the final policy (not the best checkpoint) is scored two ways, by greedy and by sampled evaluation, 20 episodes each, capped at 500.

Why both evaluations. Greedy evaluation shows what the policy's most likely action achieves; sampled evaluation shows how the stochastic policy that was actually trained behaves. A big gap means the policy is still uncertain, which matters if you deploy it stochastically, and is the analogue of temperature sampling versus greedy decoding in an LLM.

What a good report contains. The protocol, every seed's result (not just the mean), the minimum, the wall-clock cost, and the limits of what five seeds can show. The code reuses the `train` functions from Days 127-129 unchanged; the only edits were adding `if __name__ == "__main__":` guards and a `max_steps` argument to REINFORCE, so importing does not re-run those days.

Rubric (for self-marking, 10 points). Protocol stated (2); per-seed table (2); greedy and sampled reported (2); compute cost reported (1); at least one honest limitation (2); one concrete next experiment (1).

Block 2: Worked example (45 min)

Run `python code/week-19/day133_project.py` (about 4 minutes on this CPU).

Verified (out/day133_project.log):

Method	Greedy eval, seeds 10-14	Sampled eval, seeds 10-14	Time / seed
REINFORCE + value baseline	500.0, 137.5, 500.0, 500.0, 469.0	354.1, 130.2, 404.4, 360.8, 445.9	5-6 s
A2C n = 32	500.0, 498.8, 500.0, 500.0, 500.0	499.1, 485.8, 500.0, 500.0, 500.0	8-9 s
PPO clip 0.2	500.0 on all five	497.4, 500.0, 500.0, 500.0, 500.0	30-31 s

Summary lines: REINFORCE greedy mean 421.3 (min 137.5), sampled mean 339.1, 3/5 seeds at 500; A2C greedy mean 499.8 (min 498.8), sampled mean 497.0, 4/5 at 500; PPO greedy mean 500.0 (min 500.0), sampled mean 499.5, 5/5 at 500.

Reading it. At 50k steps, PPO was the most reliable (every seed at 500, greedy and almost so sampled), A2C was nearly as good, and REINFORCE was the weakest and least consistent, with one poor seed (137.5) and a large greedy-sampled gap (mean 421.3 vs 339.1), meaning its policies were still quite random. PPO cost about 4-5 times more wall-clock time than the others because of its 10 epochs of minibatch updates; per environment step it extracted more learning. Limitations: five seeds; one environment, which a 500-step cap makes easy to saturate; only the final policy scored, so the mid-run collapses seen on Days 128-129 for A2C are not visible here; hyper-parameters were the ones from earlier days, not tuned per method. Do not generalise "PPO > A2C" from a 0.2-point mean difference.

Block 3: Lab (2h)

1. Reproduce the table, then add learning curves: evaluate every 5k steps for each method (keep the total runtime to a few minutes by using 3 seeds).
2. Halve the budget to 25k steps. Does the ranking change?
3. Write a one-page report using the rubric above.
4. Next-experiment candidate: Acrobot-v1 (also in gymnasium), where rewards are -1 per step until success.

Block 4: Practice (1h)

Take the Week 19 quiz below, closed notes, 40 minutes; then mark yourself.

Block 5: Review (1h)

Do your review days. Re-mark the quiz after 24 hours and turn every wrong answer into a flashcard.

Extra notes

- Equal step budgets are fair for sample efficiency; equal wall-clock budgets would favour the cheaper updates. State which one you used.
- Fresh seeds are a cheap way to avoid tuning on the evaluation set.

Going deeper

Re-read Days 127-129 and write a one-paragraph "lab notebook" entry per method: what it estimates, what it re-uses, and how it failed this week.

Interview-style question (15 min)

"Present this table to a manager who asks which algorithm to use." Recommend PPO for reliability at this budget, mention that A2C is cheaper and close, and state the five-seed, one-environment limits.

Common mistakes

- Reporting the best checkpoint as "final".
- Re-using development seeds for the final comparison.
- Hiding the worst seed in a mean.

What to do next

Week 20 moves from RL to agents (tool use, planning, ReAct-style loops), then agent evaluation and MLOps.

Weekly quiz: Week 19 (12 questions; 40 minutes; covers Days 127-133)

1. (Day 127) Write the REINFORCE gradient and say why environment dynamics do not appear in it.
2. (Day 127) Why does a state-dependent baseline add no bias? What did a constant baseline do to gradient spread at init?
3. (Day 127) Why did the baseline runs use more environment steps, and why does that matter?
4. (Day 128) Write the one-step TD advantage. Why bootstrap on truncation?
5. (Day 128) What happened in A2C n = 1, seed 2?
6. (Day 129) Write PPO's clipped objective and explain the $A < 0$ case.
7. (Day 129) What did removing clipping do to approximate KL and to evaluation?
8. (Day 130) Rank Thompson, UCB1 and eps-greedy by regret in the log, and explain the surprise.
9. (Day 130) Why did LinUCB beat eps-greedy so clearly in the recommender?
10. (Day 131) Why did naive shaping break even value iteration, and what is potential-based shaping?
11. (Day 132) Where does the KL go in InstructGPT-style PPO and in GRPO, and why did our GRPO not reach π^*?
12. (Day 133) Summarise the project result and one limitation.

Answers

1. ?J = E[Sum_t ?log pi(a_t|s_t).G_t]; the trajectory probability factorises into policy terms and dynamics terms, and the dynamics do not depend on ?.
2. E_a[?log pi.b(s)] = b(s).?Sum_a pi = 0. Spread fell from 0.436 to 0.324.
3. They learned longer episodes under a fixed episode budget, so they saw 45,666-48,298 steps against 8,383-19,571; the comparison is confounded (fixed on Day 133).
4. ? = r + gammaV(s') - V(s). Truncation at 500 steps is not a real terminal state, so its value is not 0.
5. It collapsed to greedy eval 9.3 at 20k and 30k steps (2,117 episodes), then recovered to 500.
6. E[min(rA, clip(r, 1-eps, 1+eps)A)]. With A < 0, there is no further gain from pushing r below 1-eps.
7. Max KL rose from 0.017-0.019 to 2.352-2.790 and the clip fraction from about 0.06 to about 0.5; evaluations repeatedly collapsed to about 10 steps, and seed 1 ended at 121.
8. Thompson 30.5 < eps-greedy 112.7 < UCB1 183.3. UCB1's bonus over-explores at a 2,000-pull horizon.
9. Different users prefer different items; only LinUCB uses the context (CTR 0.876 vs 0.399).
10. The shaped MDP's optimum is to loop for repeated +0.1 rewards. Potential-based: F = gamma?(s') - ?(s), which leaves the optimal policy unchanged (but was not reliably faster: 9/10 seeds, median 38 vs 54).
11. PPO: in the per-token reward, -?.log(pi/pi_ref). GRPO: added to the loss with the k3 estimator. Std normalisation changes the reward scale and identical groups get zero advantage, so the effective ? differs: GRPO reached reward about 0.93-0.95 at KL about 2.4-2.5, against pi*'s 0.799 at 1.193.
12. At about 50k steps on fresh seeds, PPO reached 500 greedy on 5/5, A2C 4/5 (mean 499.8), REINFORCE 3/5 (mean 421.3, min 137.5). Limitation: five seeds, one easy-to-saturate environment, final policy only.